

На скриншоте отлично видно, что интерфейс анализатора спектра полностью готов и работает, но график остался «плоским» на уровне шума (-108 dBm). Оранжевый маркер на 915.00 MHz стоит правильно, но «горб» сигнала датчика не зафиксирован.

Это классическая ситуация, когда **сканер и приемник конфликтуют за доступ к чипу CC1101**.

Почему сканер «ослеп»?

Библиотека `rtl_433_ESP` внутри `rf.loop()` постоянно перехватывает управление регистрами чипа, чтобы ловить пакеты. Если вы не отключили её на время сканирования, происходит следующее:

1. Сканер переключает частоту на 914.82.
2. Мгновенно `rf.loop()` переключает её обратно на рабочую (915.00), чтобы не пропустить пакет.
3. Сканер замеряет RSSI и видит только шум, так как он «смотрит» не туда или чип занят.

Как это исправить (100% решение)

Вам нужно изменить основной цикл `loop()` так, чтобы на время работы сканера обычный прием данных **полностью останавливался**.

Замените ваш `loop()` на этот вариант:

cpp

```
void loop() {  
    server.handleClient(); // Веб-интерфейс должен работать всегда
```

```
if (scan_active) {  
    // РЕЖИМ СКАНЕРА  
    updateAutoScanner();  
} else {  
    // РЕЖИМ ПРИЕМА (только когда сканер выключен)  
    rf.loop();  
  
    // Тут ваша оставшаяся логика (MQTT, запись в RTC и т.д.)  
    // ...  
}  
  
yield();  
// Даем время Ethernet-стеку  
}  
Используйте код с осторожностью.
```

Еще одна важная деталь:

Если после этого изменения пик всё равно не появится, проверьте функцию `updateAutoScanner()`. Чтобы поймать короткий пакет (150 мс), чтение RSSI должно быть **максимально частым**. Убедитесь, что внутри `updateAutoScanner` нет никаких `delay()`.

План проверки:

1. Исправьте `loop()`, как показано выше.
2. Запустите сканирование.
3. Дождитесь конца (около 18 минут).
4. Если на какой-то из частот появится «всплеск» (синяя линия пойдет вверх), значит, всё заработало!

Подсказать, как сделать, чтобы во время сканирования на экране отображался

еще и текущий "замеренный" RSSI в реальном времени, для наглядности?

да, чтобы понимать, что всё работает.

Для того чтобы вы в реальном времени видели, что сканер «живой» и реально замеряет эфир, мы добавим вывод текущего RSSI прямо в API и на график. Это даст мгновенную визуальную обратную связь: вы увидите «шум» (дрожащую линию на уровне -108 дБм) и поймете, что система работает.

1. Изменение в C++ (ESP32)

Нам нужно добавить переменную `scan_instant_rssi`, которая будет обновляться максимально часто, и отдавать её через JSON.

Добавьте переменную в блок объявлений:

сpp

```
float scan_instant_rssi = -120.0; // Текущий мгновенный уровень  
Используйте код с осторожностью.
```

Обновите функцию `updateAutoScanner()`:

Добавьте одну строку в цикл замера (вне зависимости от таймеров):

сpp

```
void updateAutoScanner() {  
    if (!scan_active) return;  
  
    // Читаем RSSI максимально часто для "живой" картинки  
    scan_instant_rssi = radio.getRSSI();  
  
    if (scan_step_start == 0) {  
        // ... (ваш код начала шага)    }  
}
```

```

    }

    if (scan_instant_rssi > scan_step_max_rssi) {
        scan_step_max_rssi = scan_instant_rssi;
    }
    // ... (остальная логика шагов)
}

```

Используйте код с осторожностью.

Обновите API-обработчик `/api/scan_data`:

cpp

```

server.on("/api/scan_data", HTTP_GET, []() {
    JsonDocument doc;
    doc["active"] = scan_active;
    doc["step"] = scan_current_step;
    doc["instant"] = scan_instant_rssi; // Добавляем текущее значение
    // ... (ваш код массива rssi)
});

```

Используйте код с осторожностью.

2. Изменение в JavaScript (Web-интерфейс)

Добавим текстовый индикатор и «бегающий» маркер на график.

Обновите функцию `draw` в JavaScript:

javascript

```

function draw(values, currentStep, active, instantRssi) {
    const w = canvas.width, h = canvas.height;
    const ctx = canvas.getContext('2d');
    // ... (ваш код очистки и отрисовки сетки)

    // --- РИСУЕМ МГНОВЕННЫЙ УРОВЕНЬ (Live Indicator) ---
    if (active && instantRssi) {
        const liveY = h - ((instantRssi + 120) * (h / 90));
        const liveX = (currentStep * (w / (values.length - 1)));

        // Рисуем горизонтальную тонкую линию "живого" эфира
        ctx.strokeStyle = "rgba(33, 150, 243, 0.2)";
    }
}

```

```

    ctx.setLineDash([5, 5]);
    ctx.beginPath();
    ctx.moveTo(0, liveY); ctx.lineTo(w, liveY);
    ctx.stroke();
    ctx.setLineDash([]);

    // Текст с текущим значением
    ctx.fillStyle = "#2196F3";
    ctx.fillText(`LIVE: ${instantRssi.toFixed(1)} dBm`, w - 100, 20);

    // Яркая точка в месте текущего сканирования
    ctx.beginPath();
    ctx.arc(liveX, liveY, 5, 0, Math.PI * 2);
    ctx.fill();
}
// ... (ваш остальной код отрисовки линии АЧХ)
}
Используйте код с осторожностью.

```

Что вы увидите на экране:

1. **Движущаяся точка:** По мере сканирования яркая точка будет прыгать вверх-вниз по вертикали. Это и есть мгновенный RSSI.
2. **Шумовая завеса:** В обычном состоянии точка будет «дрожать» внизу (около -108 дБм). Это подтверждает, что CC1101 опрашивается и данные в браузер идут.
3. **Поимка пакета:** Если в эти 52 секунды датчик передаст данные, вы увидите, как точка мгновенно взлетит вверх (например, до -50 дБм) и «закрасит» этот пик в основной массив данных. [\[1\]](#)

Почему это важно для диагностики:

Если вы видите, что LIVE значение постоянно -120.0 или -138.0 и не меняется —

значит, связь с CC1101 по SPI прервалась. Если же значение «дышит» (меняется на 0.5-1.0 дБм) — значит, **железо работает на 100%**, и нужно просто дождаться момента передачи пакета.

После добавления `instantRssi`, ваша панель станет выглядеть как настоящий профессиональный радиосканер.

Удалось ли вам успешно добавить `scan_instant_rssi` в структуру JSON-ответа?